

Object Detection

What could be an architecture to do this task?

A straightforward approach to this task is training a Convolutional Neural Network (CNN) with a multi-headed output: one head for classification and another for localization.

The Workflow:

- Train a CNN model.
- Extract the flattened layer feature map.
- Pass the features simultaneously through the classification and localization heads.
- Verify that the classification and localization outputs correspond correctly to the same object.

The basic CNN architecture mentioned above, YOLO (You Only Look Once), and SSD (Single Shot MultiBox Detector) all fall under **Single-Stage Detector Architectures** simply because they detect the object in a single forward pass.

Note: Single-stage detector architectures are fast, but they can struggle with precision, particularly on small or overlapping objects.

How Do We Prioritize Precision?

Single-stage detectors often struggle with precision, which indirectly creates a lot of false positives. To overcome the problem of false positives, you can implement the following strategies:

1. **Control the Confidence Level** You can filter out low-certainty predictions by raising the model's confidence threshold. By demanding a higher confidence score before a bounding box is displayed, you eliminate noisy, incorrect detections.
 2. **Add a Rechecking Head (Two-Stage Architecture)** You can introduce a dedicated "rechecking head" by moving to a two-stage detector architecture (like Faster R-CNN).
 - **Stage 1:** Acts as a quick filter to propose potential object regions.
 - **Stage 2:** Acts as the rechecking mechanism, meticulously evaluating only those proposed regions to confirm exactly what and where the objects are.
-

What can we do to improve the previous architecture?

The previous architecture we designed can track only up to one object in an image.

What if we had more than one object?

The primary limitation of a standard global regression architecture is its inability to detect multiple objects simultaneously, as it is mathematically constrained to output a single set of coordinates per image.

To resolve this, we can discretize the input image into a localized grid of smaller, uniform sub-regions (or "cells"). By shifting the network's objective from a single global prediction to multiple localized predictions, each individual cell becomes responsible for detecting at most one object whose center falls within its boundaries. If a sub-region contains no object, it is simply classified as background. This grid-based approach effectively transforms a complex, multi-object detection task into parallel, single-object regression problems.

How do we create Grid Division?

Using a Convolutional Neural Network (CNN) for object detection allows us to predict a specific number of variables per grid cell. The total number of predicted variables equals:
$$\text{Total Variables} = 1 \times (\text{Object Confidence}) + 4 \times (\text{Bounding Box Coordinates}) + N \times (\text{Number of Classes})$$

According to your configuration file, the dataset has exactly $N=6$ classes (nc: 6). Substituting this into the formula gives:
$$\text{Total Variables} = 1 + 4 + 6 = 11$$

Instead of predicting one global set of coordinates for the entire image, the convolutional backbone processes the image while maintaining spatial conservation, effectively treating the output as a grid of smaller localized regions. This allows us to construct an output tensor of shape:
$$\mathbf{(S, S, 11)}$$

In this architecture, the channels inside each individual (x, y) grid cell represent the object's presence, its local bounding coordinates, its dimensions, and its specific classification probabilities.

Channel Mapping Example

For your model trained to detect these 6 specific classes, the channel breakdown for every single cell in the $S \times S$ grid maps out exactly like this:

- **Channel 0 $\rightarrow$ Object Confidence:** The probability that an object's center falls inside this specific grid cell (bounded between 0.0 and 1.0).
- **Channel 1 $\rightarrow$ Box Center X (x_c):** The horizontal center of the bounding box, calculated relative to the boundaries of the current cell.
- **Channel 2 $\rightarrow$ Box Center Y (y_c):** The vertical center of the bounding box, calculated relative to the boundaries of the current cell.

box, calculated relative to the boundaries of the current cell.

- **Channel 3 $\rightarrow$ Box Width (\$w\$):** The total width of the bounding box, scaled relative to the dimensions of the entire image.
- **Channel 4 $\rightarrow$ Box Height (\$h\$):** The total height of the bounding box, scaled relative to the dimensions of the entire image.
- **Channel 5 $\rightarrow$ Probability:** Pistol
- **Channel 6 $\rightarrow$ Probability:** Smartphone
- **Channel 7 $\rightarrow$ Probability:** Knife
- **Channel 8 $\rightarrow$ Probability:** Monedero (purse/wallet)
- **Channel 9 $\rightarrow$ Probability:** Billete (banknote/bill)
- **Channel 10 $\rightarrow$ Probability:** Tarjeta (card)

Spatial Overlap Resolution By organizing the channels this way, the final 1×1 convolutional layer acts as an array of parallel detectors. If a person is holding a **smartphone** in their hand (activating a grid cell in the center) while a **tarjeta** is sitting on a table in the bottom-right corner, the corresponding spatial cells will activate and output their respective coordinates and probabilities simultaneously without interfering with one another.

Note: Convolutional layers preserve spatial data, making the above method analogous to creating grids.

Why do we predict the box center?

We have different objects in our image, but imagine a grid cell that has **50%** of class 0 and **50%** of class 1. What will the grid cell be forced to predict? Also, note that we predict box width and box height relative to the size of the image, so forcing a grid cell to predict the center is more practical than forcing individual cells to predict individually.

The box width and height are predicted using the sigmoid function. You might note that this can cause the vanishing gradient problem, because the derivative is:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

The maximum value of this function can be **0.25**. This causes the vanishing gradient problem in the architecture.

Anchor Boxes (Handling Scale Diversity)

If a single cell is responsible for a giant object, how does it know how to calculate those massive shapes cleanly without its gradients exploding or vanishing? It uses **Anchor Boxes** (or prior boxes).

Instead of making the network guess box dimensions completely from scratch, engineers look at the training dataset beforehand and calculate the most common object shapes (e.g., small square boxes for finger fractures, tall skinny boxes for forearm fractures, and giant wide boxes for shoulders).

Every grid cell is given a set of these pre-defined shapes as templates:

- **Anchor 1:** Small Square (Fingers)
- **Anchor 2:** Medium Vertical Rectangle (Forearm)
- **Anchor 3:** Large Horizontal Rectangle (Shoulder/Humerus)

Instead of predicting raw sizes, the cell simply predicts a scaling factor to tweak the closest-matching anchor template:
$$\text{Final Width} = \text{Anchor Width} \times e^{\text{predicted scale}}$$

If a massive object is present, the cell naturally selects its largest anchor template and scales it up slightly, allowing it to easily capture objects much larger than the cell itself.

Non-Maximum Suppression (NMS)

Because large objects cover so much territory, a side effect occurs: multiple adjacent grid cells might get confused and all attempt to predict a bounding box for the same giant object. To clean up this mess, **Non-Maximum Suppression (NMS)** filters the final outputs at the very end of the pipeline.

NMS operates through the following algorithmic steps:

1. **Identify the Highest Confidence Box:** It reviews all predicted boxes across the entire grid and selects the one with the absolute highest **Object Confidence** score.
2. **Measure the Overlap (IoU):** It calculates how much the other neighboring boxes overlap with this top-performing box using a metric called **Intersection over Union (IoU)**:
$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$
3. **Suppress the Duplicates:** If a nearby box overlaps with the best box by more than a pre-defined threshold (e.g., more than **50%** overlap), NMS assumes they are targeting the same object and aggressively deletes the weaker box.
4. **Repeat:** This loop repeats for the remaining boxes until no overlapping duplicates are left.

This process eliminates the clutter, leaving you with exactly one clean, perfectly fitted bounding box around the targeted object